

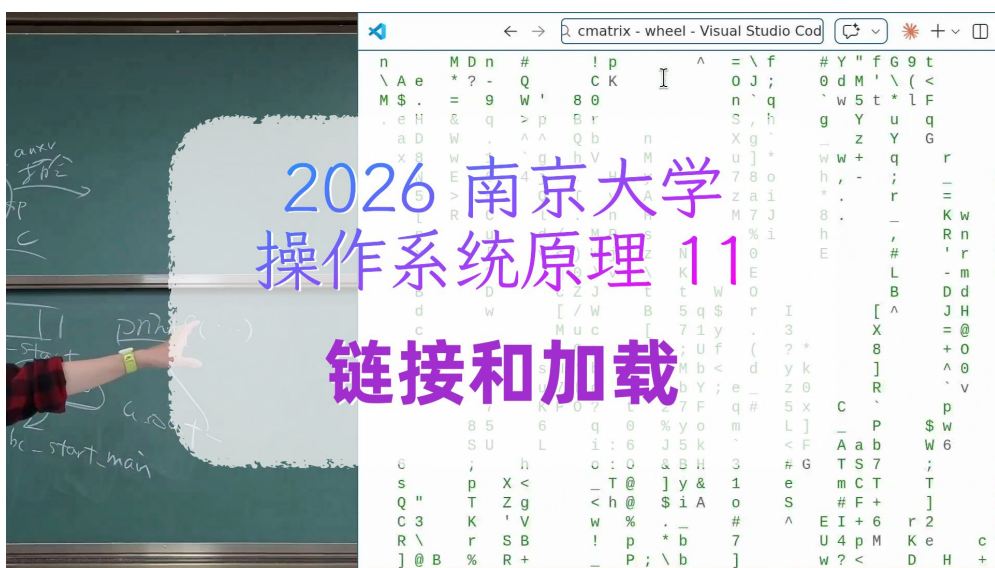
课程笔记

链接和加载

南京大学 操作系统原理 2026 春季学期 第 11 讲

lecture-to-notes

2026 年 4 月 17 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 29 分钟

视频链接：<https://www.bilibili.com/video/BV1CoDzB5Eey>

目录

1 引入：从系统调用到链接加载	2
2 可执行文件：进程初始状态的描述	2
2.1 从 file /bin/ls 说起	2
2.2 更精确的定义：进程初始内存布局	3
2.3 a.out：最简 ELF 的远祖	3
2.4 ELF 难读的根本原因：为机器设计	4
3 Funny Little Executable：我们总可以在等价描述形式之间转换	5
3.1 想法的起点：把 offset 指针全部去掉	6
3.2 再进一步：用 Markdown 做 ELF	6
3.3 重定位的”人话版本”：message - . - 4	7
3.4 这还没完：SVG、Emoji 都可以	8
4 链接和加载的真正含义	8
5 加载器源码：binfmt_elf.c 与 AI 阅读内核	9
6 Shebang：隐藏在加载器里的”第二格式”	10
7 从静态链接走向动态链接	11
7.1 静态链接的两大问题	11
7.2 动态链接的运行时开销是真的共享吗？	12
8 动态链接的实现：解释器 (ld-linux) 与 PLT/GOT	13
8.1 PT_INTERP：ELF 的”解释器”字段	13
8.2 PIC / PLT / GOT：运行时重定位的代价	14
9 LD_PRELOAD：动态链接赋予的”超能力”	15
10 总结与延伸	16
10.1 讲师的收束	16
10.2 本讲的核心压缩	17
10.3 拓展阅读与动手	17

1 引入：从系统调用到链接加载

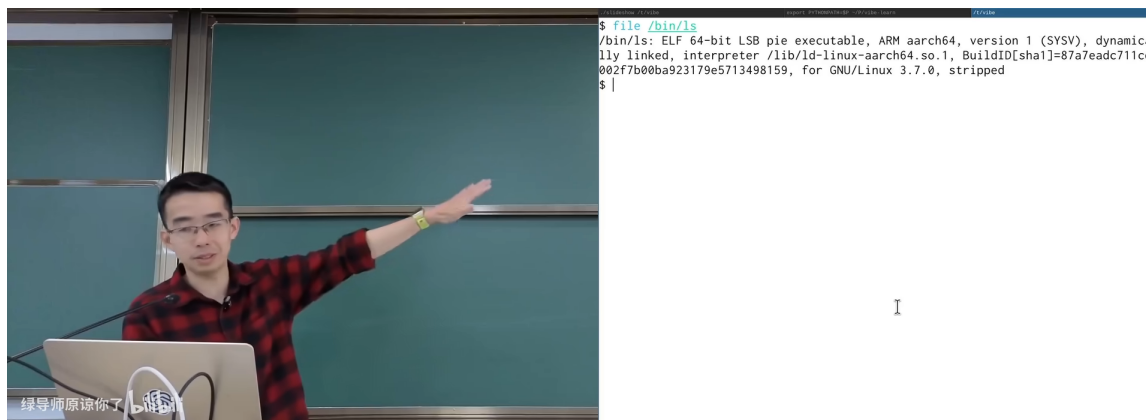


图 1: 本讲脉络：链接和加载本质上仍基于系统调用（`execve + mmap`），只是为**进程初始状态**的数据结构加了描述层²

前面几讲，课程一直围绕**系统调用**这一接口展开：用 `fork/execve/exit` 管理进程，用 `open/read/write` 操作文件描述符，用 `mmap/munmap` 管理地址空间。今天讲的**链接和加载**，看起来是一个独立的话题，但它本质上**仍然建立在系统调用之上**——你也完全可以只用系统调用从零实现一个链接器和加载器。

一句话把握今天的核心

可执行文件是描述进程初始状态的数据结构。链接器的工作是把若干**片段化的初始状态**（object file）合并成一份**完整的初始状态描述**（executable）；加载器的工作是把这份描述**搬到内存里**，然后交给 CPU 开始执行第一条指令。所谓 ELF、PLT、GOT 这些繁琐术语，都只是这个核心想法在不同角度的投影。

2 可执行文件：进程初始状态的描述

2.1 从 `file /bin/ls` 说起

对可执行文件最朴素的理解是双击可以弹出窗口的 **Steam**、**原神** 启动的 **.exe**。学完操作系统之后，这个认识应该深化：**可执行文件首先是操作系统里的一个普通文件**，可以 `cat`，可以 `hexdump`，甚至可以用任意文本编辑器打开。

² 视频画面时间区间：00:01:00–00:01:15。内容：jyy 承接前面的 `fork/execve/mmap` 等系统调用，引出今天的 ELF 链接和加载。

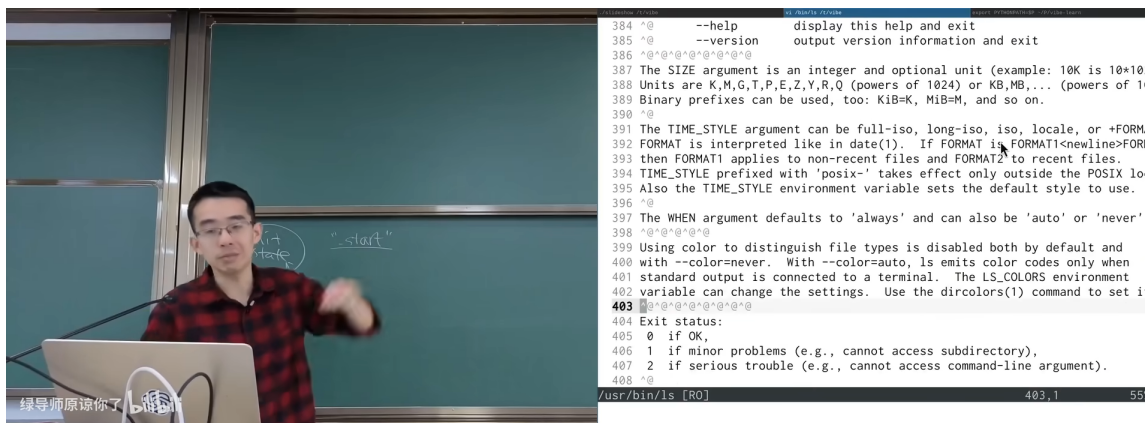


图 2: `file /bin/ls` 返回 ELF 64-bit LSB executable; `hexdump` 可以看到文件头的魔数 7F 45 4C 46 (`\x7f ELF`), x86 小端存储⁴

0x7F ELF 从哪来？

0x7F 是 ASCII 的 DEL (DELETE) 字符，故意选一个非可打印、不会与常见文本格式冲突的字节做起始魔数；其后三字节是 ASCII E/L/F。这样设计的好处是：内核读到前四个字节，只要匹配 7F 45 4C 46 就知道“这是一个 ELF 文件”，然后按 ELF 规范继续解析后续字段。下一节我们会看到脚本 `#!` 也是类似的“头两个字节判断”机制。

2.2 更精确的定义：进程初始内存布局

可执行文件

所谓“可以执行”，是因为操作系统可以根据这个数据结构准备一份初始进程状态——包括 Initial Process Stack (System V ABI 规定: `argc / argv[] / envp[] / 辅助向量 auxv[]`)、若干段代码数据的加载位置、权限，等等。当 PC 指向程序入口时，一切已经就位。换句话说：可执行文件既不是二进制的指令堆，也不是抽象意义上的“程序”，而是一份结构化的“开机预案”。`execve` 把这份预案交给内核执行。

从这个定义出发，下面两件事顺理成章：

- 可执行文件内部的字段 = 进程初始状态的各个部分（代码段、数据段、BSS、入口地址、解释器、符号表、重定位表 ...）
- 加载 = 把这些字段按权限映射到进程地址空间（通过 `mmap` 系统调用）
- 链接 = 把多份“片段化初始状态”合并成一份完整初始状态（本质是“算数字填地址”）

2.3 a.out: 最简 ELF 的远祖

在深入 ELF 前，先看 Unix `a.out` (Assembler output) ——这个格式因为极简单，反而最能说明“可执行文件是初始状态”这个本质。

⁴视频画面时间区间：00:09:40–00:09:55。内容：`hexdump -C /bin/ls` 看到 7F 45 4C 46 (ELF 文件头)。讲师现场用 ARM 机器演示，但原理在 x86 上完全一致。

struct exec: 整个文件头只有几个字段

a.out 的核心就是一个 C struct: a_magic (魔数)、a_text (代码长度)、a_data (数据长度)、a_bss (BSS 长度)、a_syms (符号表大小)、a_entry (入口地址)、a_trsize/a_drsize (重定位表大小)。**甚至没有 offset 字段**——代码数据加载到内存哪里，完全由平台 ABI 常数决定，硬编码在操作系统内核里。

a.out 显得简单，但正因如此它**不支持动态链接、没有调试信息、没有内存对齐控制、没有 thread-local storage**——所有这些现代特性全部被加进了 ELF。所以理解 a.out → 理解 ELF 的”祖先形态” → 理解为什么 ELF 长成今天这样。

2.4 ELF 难读的根本原因：为机器设计

为什么 ELF 让人”换字典似的吐血”？

所有学过《计算机系统基础》的同学对 ELF 这一章都印象深刻——要背 program header、section header、各种 PT_LOAD / PT_DYNAMIC、R_X86_64_PC32 重定位类型...讲师说他甚至做过调查：**每个上过这门课的同学都吐过血；两位讲这门课的老师也吐过血。**根本原因是：**ELF 是为机器的效率设计的**，而不是为人类可读设计的。它大量使用：

- **offset 指针**：一个字符串不直接写在这里，而是”文件第 4587 字节处”。机器加一个偏移就读到，人脑算不动。
- **bitfield 编码**：代码段的权限用 R|X，数据段用 R|W，全部压成 bit 位。
- **扁平表格**：所有符号、重定位堆在一个表里，靠索引串起来。

这些设计对机器友好（一次内存访问就能跳转），但必须”翻译成一张平坦结构的纸”人才能理解——而这正是教材和 slides 反复做的苦差事。

3 Funny Little Executable: 我们总可以在等价描述形式之间转换

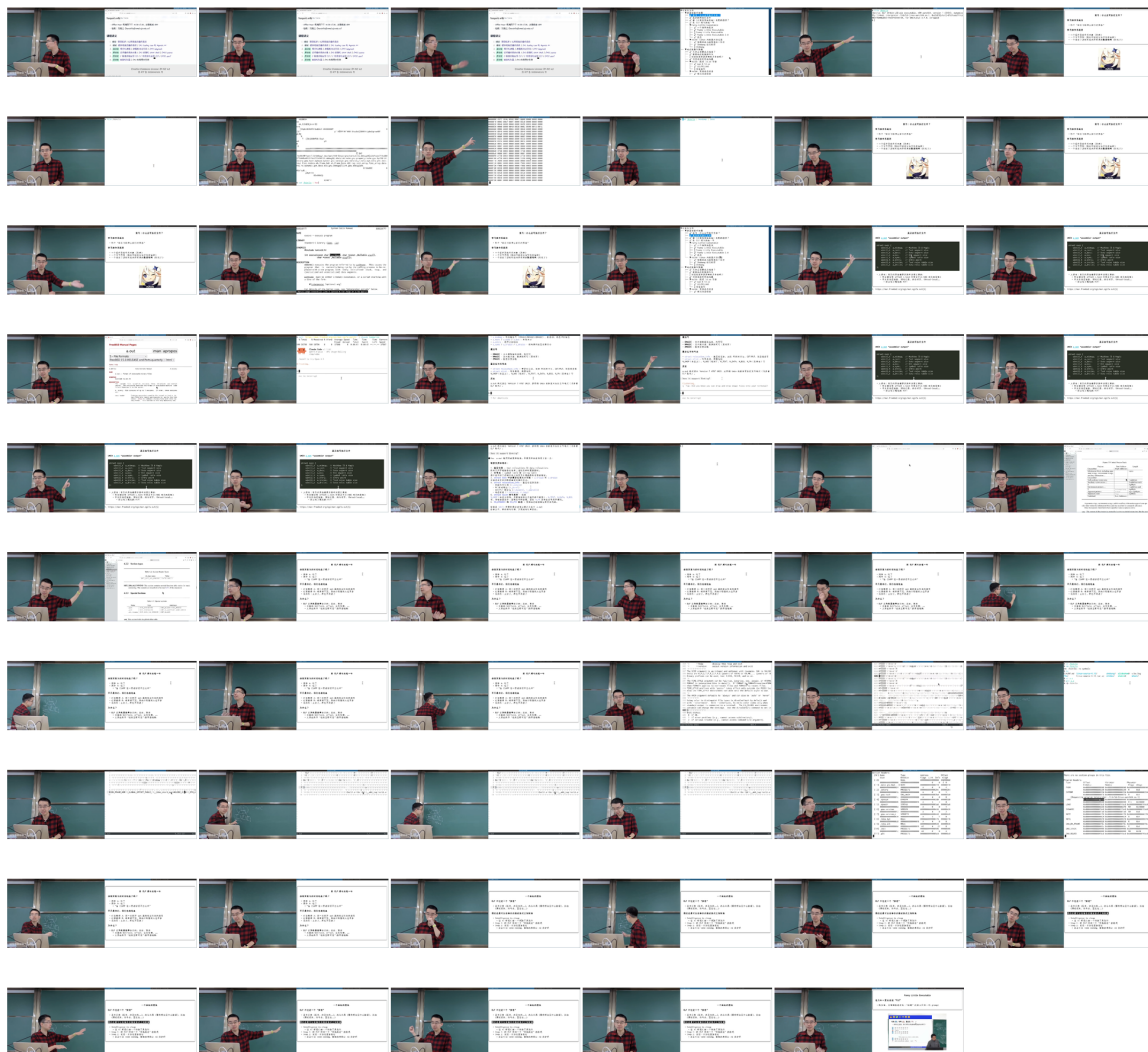


图 3: Funny Little Executable: 用 JSON 重新描述”进程初始状态”, 保留所有教学信息量, 却去掉了所有 offset 指针⁶

课程贯穿全程的一条基本原理

我们总可以在等价的描述形式之间转换。

一个数据结构本质上是若干字符串 + 指针 + 字段的组合。ELF 是一种描述, a.out 是另一种描述, 我自己设计的 JSON / Markdown / SVG / Emoji 图形...都是等价描述。只要不丢失信息, 选一个对你当前任务最方便的表示就好。

⁶视频画面时间区间: 00:14:40–00:14:55。内容: JSON 文件描述一个.exe, 包含 symbol __start 在第 15 位置、指令序列、libc 链接信息等。

3.1 想法的起点：把 offset 指针全部去掉

”我愿意用空间换可读性”

讲师在准备这节 slide 时的一个关键思考：

”我需要任何信息的时候，我可以以浪费空间的方式来换取好理解。”

于是他给 AI 提出的设计 requirements 是：把 ELF 里所有 offset 形式的指针结构全部去掉。字符串要直接写在引用它的位置，符号要直接写出名字而不是表索引，权限要直接写 RIX 而不是 bitfield。以浪费空间换取每一处都能”字面阅读”。

这个”等价描述”想法的实际落地就是 Funny Little Executable：

- 头部不用 7F ELF 魔数，直接写 .exe 或 "type": "executable"；
- 符号不用 offset 指针，直接写 _start 在第 15 字节；
- 指令段直接列出十六进制字节，每条指令前加一个符号标记”这是代码”；
- 整套格式还自举出了完整的编译器工具链——把 C 语言直接编译成 Funny Little Executable，然后加载运行，打印出 Hello。这一切都是让 AI vibe code 完成的。

讲师多年维护这个 demo 的反思

Funny Little Executable 讲师已经改进了好几年。今年他有一个颇为感慨的观察：

”如果我要改一个设计，我现在有一个新的想法，我马上就能把它实现出来——因为我可以 *vibe coding*，我只要把我的设计说出来，它就可以帮我实现。也就意味着，我这么些年以来所有聪明的努力，可能现在都不那么必要了。”

从前”提出新格式”要花一学期写 demo；2026 年，一句话可以替代。教学者的稀缺价值从”精巧实现”转向”提出值得被实现的好想法”。

3.2 再进一步：用 Markdown 做 ELF

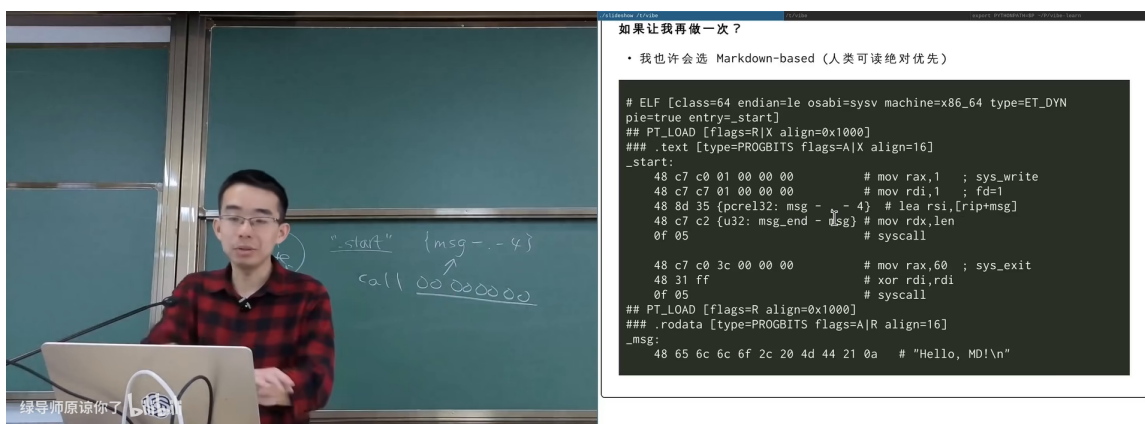


图 4：用 Markdown 表达 ELF：属性写进中括号，PT_LOAD 段嵌套 .text 节，节里写符号和指令，重定位直接展开成”要填哪个数字”的表达式⁸

这份 Markdown 设计的要点：

- ELF[64-bit, little-endian, System V, x86_64]：顶层属性全部摊平写在中括号里，不用 bitfield；
- PT_LOAD[R|X]：PT_LOAD 段直接写权限，读/执行不可写；
- 嵌套结构 .text：PT_LOAD 里直接嵌套 .text 节；
- _start：.text 里直接列出符号名和对应的机器码字节序列；
- 重定位不再神秘——在需要填数字的地方直接写表达式，后面会看到。

3.3 重定位的”人话版本”：message - . - 4

R_X86_64_PC32 不需要记名字，只需要知道”填哪个数字”

原版教材讲重定位时，学生要记各种重定位类型：R_X86_64_PC32、R_X86_64_GOT32、R_X86_64_PLT32、R_ARM_CALL...记得头大。

Markdown ELF 的做法是：直接把这个类型翻译成一个算术表达式。比如 R_X86_64_PC32 的含义就是”在这 4 字节里填 message - . - 4”——

- message：目标符号的地址
- . (dot)：当前填入位置的地址
- -4：因为 x86 的 PC 相对寻址是相对于下一条指令的开头，而占位的 4 字节刚好在指令末尾

这三个量在链接时全部已知，减一下得到一个 32 位整数，填进去就完事。整个重定位类型表变成了”填什么算式”的常识。

call 指令的运行期故事

具体到 call foo：

1. 编译 a.c 得 a.o。此时你写的 call foo 里的 foo 其实不知道在哪——甚至 foo 是否真的存在编译器都不管。
2. 编译器发射一条 call 指令 (opcode E8) 后跟四个字节的 0 (占位)，同时在重定位表里记一行：”这里要填 foo - . - 4”。
3. 链接器看到所有 .o，为每个符号分配最终地址。遍历重定位表，foo 的地址减去当前 . 再减 4，得到一个 32 位偏移量，填进刚才的那四个 0。
4. 如果 foo 从头到尾没有定义，链接器报 undefined reference。

这就是”链接”。就这么一件事。

⁸视频画面时间区间：00:22:10-00:22:30。内容：AI 给出的 markdown 形式 ELF 设计。

加载也只是说一句 mmap

Markdown 里 PT_LOAD[R|X] 字面意义是什么？”把这个段以**可读可执行**的方式搬到内存”。这就是一句 `mmap(vaddr, size, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED, fd, offset)`。没有其他神秘性。所谓的”加载器”不过是循环遍历所有 PT_LOAD 段，依次 mmap。

3.4 这还没完：SVG、Emoji 都可以

讲师在 slide 最后点明：JSON、Markdown 都不是唯一选择。

- **SVG 版本**：把 ELF 画成图形，每个段是一个方框，符号是箭头。**链接两个 SVG 就是把两张画布拼起来**——完全可以用 SVG 来实现链接和加载，只要你能跑得动。
- **Emoji 版本**：用 等表情符号把 ELF 的意思画出来，再配少量文字解释。
- **核心点**：你可以有**任何一种你喜欢的描述形式**，只要信息等价，剩下的工作交给 AI 把 ELF 翻译过去就好。

AI 时代的”等价描述”自由度

讲师在课上坦言：**以前花大量时间做可视化 demo 可能不必要了**。设计一个合适的”等价描述”后，让 AI 帮你写脚本把 ELF 翻译成这种描述，代价几乎为零。更有趣的是 AI 可以帮你**枚举各种描述形式**——每种都能产生一个”Funny Little ELF”变体。**理解原理比掌握 ELF 细节更重要**。

4 链接和加载的真正含义

经过上面的重新描述，链接和加载各自归约成一件极其简单的事：

链接

在编译阶段，如果你写 `call foo`，编译器无法知道 `foo` 的最终地址——它只能发射出一条 `call` 指令加**四个字节的占位 0**，同时在**重定位表**里记录：“这里需要填 `foo` 相对于当前位置的偏移”。

链接时，所有目标文件合并，每个符号都有了确定地址，重定位器**遍历重定位表**，算出 `foo - . - 4` (**32 位整数**)，填进那四个字节。完事。

加载

每个 PT_LOAD 段都有 `vaddr` (虚拟地址)、`filesz/memsz` (文件/内存长度)、`flags` (权限)。加载器就是一句 `mmap(vaddr, memsz, prot=flags, MAP_PRIVATE|MAP_FIXED, fd, offset)`——把文件某段按指定权限映射到指定虚拟地址，仅此而已。

之前一学期的课支配你恐惧的所谓”链接器”、”加载器”，抽象掉数据结构后就是这两句话。

5 加载器源码：binfmt_elf.c 与 AI 阅读内核



AUXV 类型	值	说明
AT_HWCAP	ELF_HWCAP	硬件能力标志（由架构定义）
AT_PAGESZ	ELF_EXEC_PAGESIZE	系统页面大小
AT_CLKTCK	CLOCKS_PER_SEC	每秒时钟滴答数
AT_PHDR	phdr_addr	程序头表在内存中的地址
AT_PHENT	sizeof(struct elf_phdr)	每个程序头的大小
AT_PHNUM	exec->e_phnum	程序头数数量
AT_BASE	interp_load_addr	动态链接器（interpreter）的加载基址
AT_FLAGS	flags	标志位（如 AT_FLAGS_PRESERVE_ARGV0）
AT_ENTRY	e_entry	程序入口点地址
AT_UID	cred->uid	真实用户 ID

绿导师原谅你了
© Kkxiao Code

图 5: Linux 内核 fs/binfmt_elf.c 的关键流程: 检查数据结构合法性 → 加载 program headers → 映射 LOAD 段 → create_elf_tables 在栈上布置 argc/argv/envp/auxv¹⁰

auxv 和 vdso 的关系

auxv (辅助向量 **auxiliary vector**) 位于 Initial Process Stack, 在 **envp[]** 之后。这是一个 (type, value) 对的列表, 传递内核想告诉用户程序的信息。

其中最有意思的一项是 `AT_SYSINFO_EHDR`——**vdso 的入口地址**。`vdso` (virtual Dynamic Shared Object) 是一段内核映射到每个进程地址空间的代码，允许诸如 `gettimeofday` 这类“只读系统信息”的系统调用**不进内核直接在用户态执行**，大幅减少开销。具体插入点在 `ARCH_DLINFO` 宏中 (`arch/x86/entry/vdso/vma.c` 附近)。

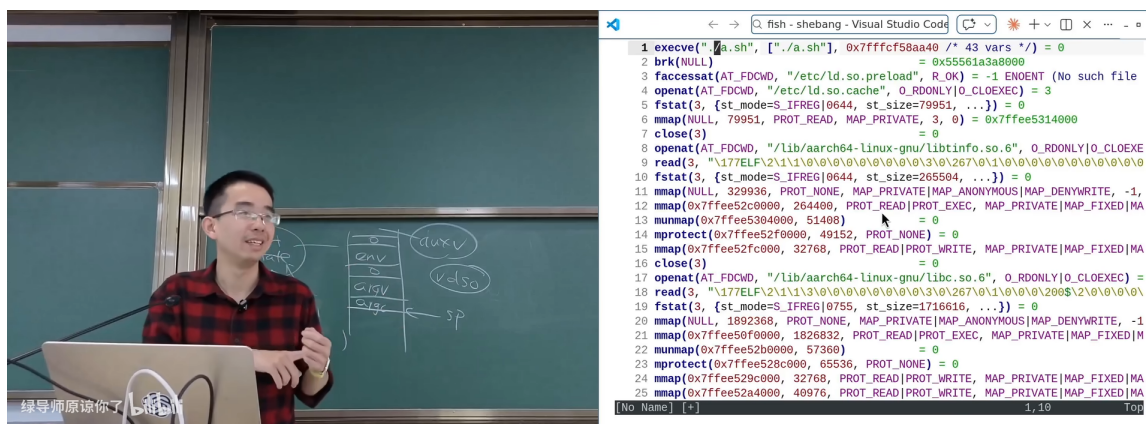


图 6: Initial Process Stack 的布局: RSP 指向 `argc`, 其上是 `argv[]` 指针数组、`envp[]`、`auxv[]`, 顶部是实际的字符串数据¹²

¹⁰ 视频画面时间区间:00:32:15-00:32:45。内容:AI 读 binfmt_elf.c(2185 行),分块阅读后总结出关键流程,并定位到 `create_elf_tables` 函数在栈上布置 Initial Process Stack。

¹²视频画面时间区间: 00:35:00~00:35:30。内容: slide 展示 Initial Process Stack 结构, 包括 SP 指向位置、argc 数量、argv 向量、envp 环境变量、auxv 辅助向量。

AI 让读内核从“劝退”变“可行”

九万个文件的 Linux 内核代码库，以前新手点开就劝退。但用 Agent 只需要一个 prompt——“请读 fs/binfmt_elf.c 告诉我加载器做了什么”——AI 就会分块（400 行 400 行地）读文件、跨文件跳转、定位到 ARCH_DLINFO 宏在 x86 下的定义，然后给你一个比老师讲得还清楚的 summary。

讲师的一句感慨：“Agent 面前你只需要能问出这个问题。你不需要对 codebase 有任何熟悉。”2023 年 Claude 还做不到这个；2026 年（Opus 4.7 之后）已经是常规操作。

6 Shebang：隐藏在加载器里的”第二格式”

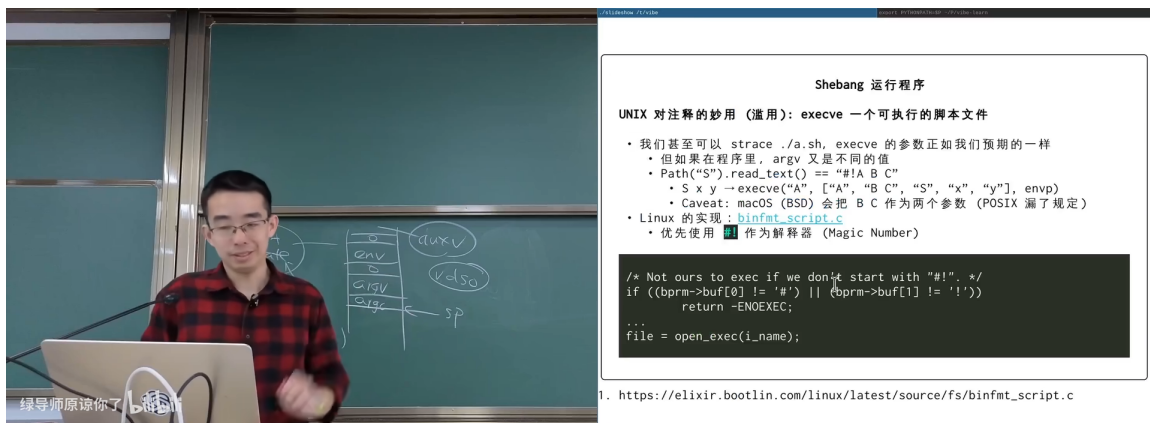


图 7: Shebang 执行流程：内核读文件头两字节 `#!`，提取解释器路径和参数，改为执行”解释器 + 原脚本”¹⁴

Shebang 由内核加载器实现，不是 shell 特性

很多人以为 `#!/bin/bash` 是 shell 的魔法。其实这是操作系统内核加载器的硬编码逻辑：`fs/binfmt_script.c` 检查 `buf[0]=='#' && buf[1]=='!'`，成立则把”解释器路径 + 原文件”作为新的 `argv` 重新 `execve`。

所以 `execve` 能执行的不只是 ELF：

- 以 `#!` 开头的任何文本文件 → 当脚本加载
- ELF 格式的二进制文件 → `binfmt_elf.c` 处理
- (早期 BSD) a.out 魔数 → `binfmt_aout.c` 处理

POSIX 留下的 Shebang”彩蛋”

POSIX 没规定 `#!` 第一行的参数该怎么拆：`#!/bin/./a b c` 在 Linux 把 `b c` 合并为一个参数传给 `a`；macOS 把 `b c` 当两个独立参数传。所以跨平台脚本里 shebang 的参数要慎用。另一个副作用：如果一个文件不是合法 ELF 但以 `#!` 开头，加载器仍会试图执行它——按第一行的路径作为解释器。于是 `execve` 上出现的 `"/bin/./a.txt"` 也能正常跑。

¹⁴视频画面时间区间：00:42:00-00:42:30。内容：slide 展示 `#!/bin/bash` 类似的机制。内核 `fs/binfmt_script.c` 硬编码检查 `buf[0]=='#'` 和 `buf[1]=='!'`，为真则按脚本加载。

7 从静态链接走向动态链接

7.1 静态链接的两大问题

静态链接的代价

早期 `a.out` 不支持动态链接，所有二进制都把 `libc` 完整拷贝一份进自己的 `.text` 段。问题立竿见影：

问题 1 — 磁盘空间浪费。 `/usr/bin` 下随便就有 2100 个可执行文件。如果每个都静态链接一份 2 MB 的 `libc`，那就是 4 GB 纯冗余。

问题 2 — 安全补丁难以推送。 假设明天 `libc` 发布一个关键安全补丁，静态链接的世界意味着要重新编译并分发全部 2100 个程序。这种模式不可持续。

动态链接的解法：把 `libc` 从二进制里剥离，放到单独的 `libc.so` 文件，所有进程运行时再加载同一份 `libc.so`。既省空间又能单独升级。

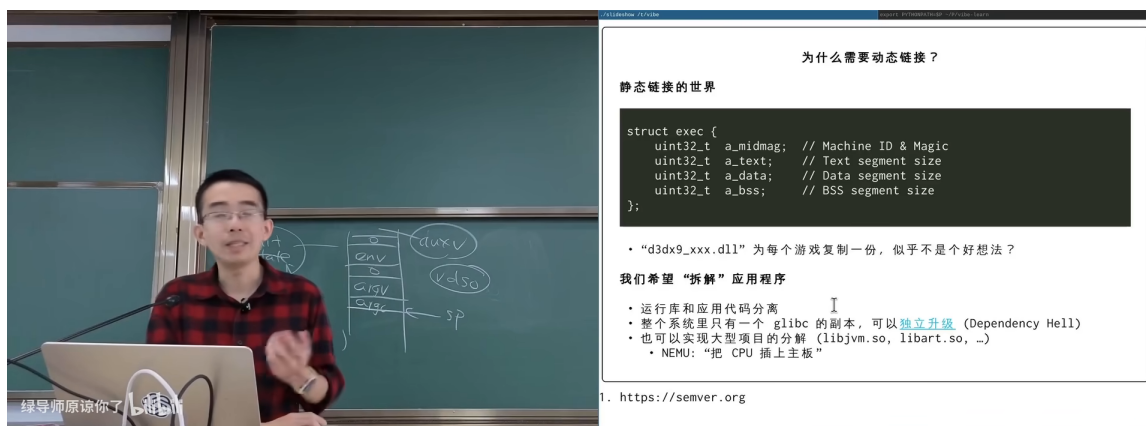


图 8: 动态链接的动机：共享库的磁盘 / 内存共享以及组件化（NEMU 可以拆成 `nemu.so` + 主板框架，实现 differential testing）¹⁶

¹⁶视频画面时间区间：00:46:00–00:46:20。内容：slide 展示动态链接的好处——空间节约、热更新、以及作为组件化机制（用户 CPU 实现可编译为 `.so` “插”到模拟器里）。

7.2 动态链接的运行时开销是真的共享吗？

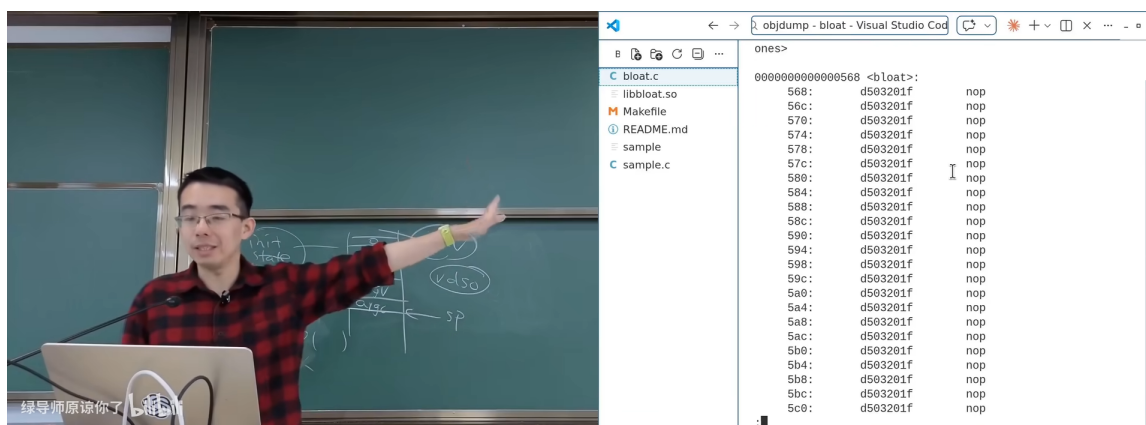


图 9: pmap 查看 fish 进程地址空间: libc.so、libm.so、libgcc.so、libpcre2.so（正则表达式库）等被映射进来，各有 R/X 代码段和 R/W 数据段¹⁸

mmap + MAP_SHARED 让内存里也只有一份

虚拟内存给动态链接一个“额外赠礼”：无论启动多少个进程，同一个 libc.so 在物理内存里只有一份。只读的代码段和只读数据段通过 page table 映射到每个进程的地址空间，写时复制。

如何验证？讲师设计了一个 libbloat.so + 1000 个 sample 进程的实验：

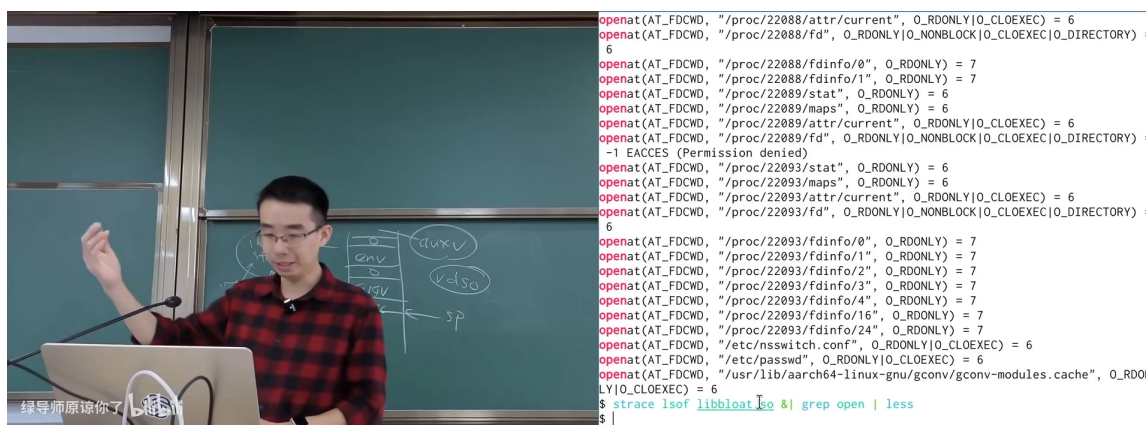


图 10: libbloat.so 实验：库里嵌入 10 MB 的 NOP；启动 1000 个链接它的 sample 进程。系统只用 4.7 GB 内存，而非预期的 40 GB，证明 .text 在进程间共享²⁰

¹⁸ 视频画面时间区间：00:49:50–00:50:05。内容：终端命令 pmap 17644 (fish shell 的 PID) 输出进程地址空间映射。

²⁰ 视频画面时间区间：00:56:00–00:56:30。内容：终端里 1000 个 sample 进程运行；free -h 显示系统内存使用不过 4.7 GB（包括 VS Code / Claude Code 等），如果每进程独立 10 MB 则应耗 10 GB 以上。

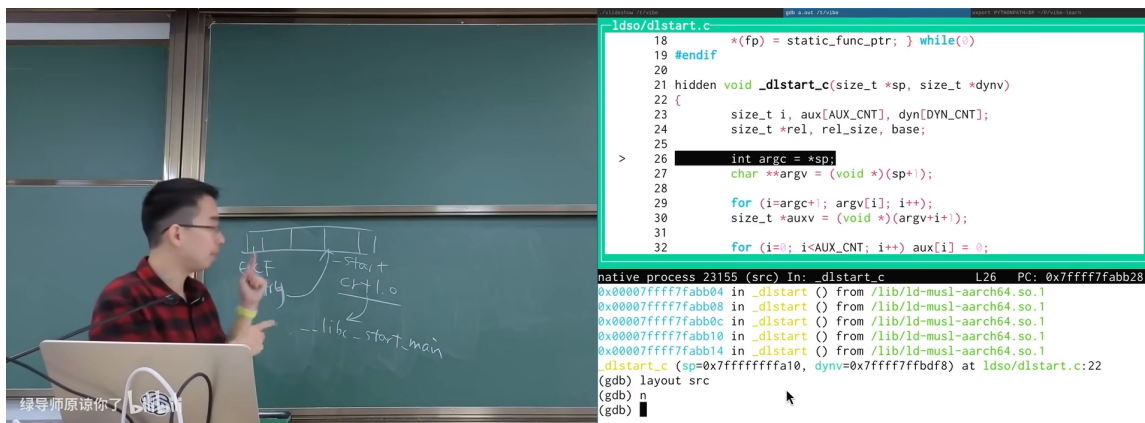


图 11: lsof libbloat.so 找到 1001 个打开者 (1000 个 sample 进程 + 1 个 lsof 自己)。lsof 的实现原理: 遍历 /proc/[pid]/maps, 查找命中的文件²²

“把工具”搭”出来 vs. 记住命令

讲师强调一个学习方法: ”精心设计的实验背后是精心设计的思路”。pstree 实验让你熟悉 procs; 于是学到动态链接时你能自己想到 /proc/[pid]/maps 能查库映射; 遇到”有多少人用这个库”的问题时你能自己想到 lsof 背后其实就是扫 procs。

每一个”神奇的命令行工具”不过是你已经知道的底层机制包装了一层。一旦理解这一点, 你不需要记任何命令——需要时让 AI vibe code 一个出来就行。

8 动态链接的实现: 解释器 (ld-linux) 与 PLT/GOT

8.1 PT_INTERP: ELF 的”解释器”字段

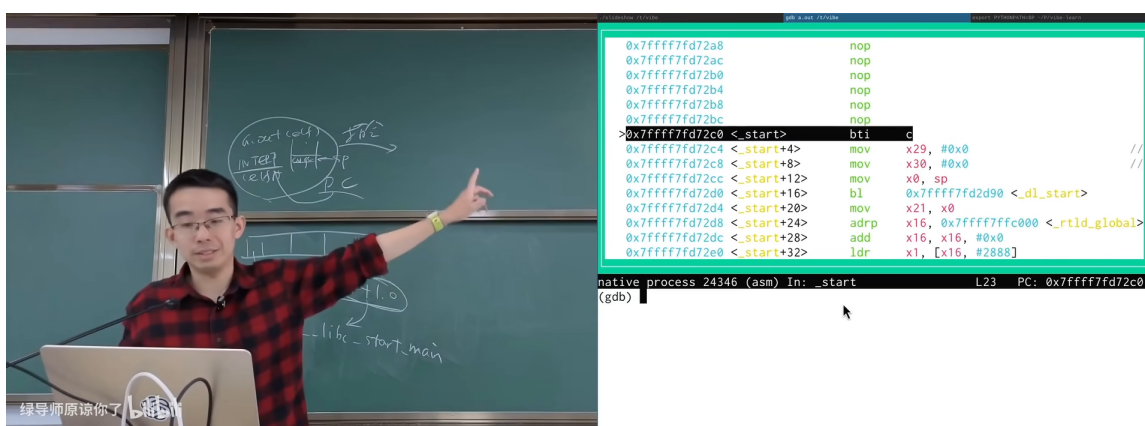


图 12: readelf -a 看到 Requesting program interpreter: /lib/ld-musl-aarch64.so.1; 这个字符串可以直接用十六进制编辑器修改²⁴

²²视频画面时间区间: 01:02:15–01:02:45。内容: lsof /path/libbloat.so 输出一千多个进程 PID。strace -f lsof | grep open 证实它只是在扫 procs。

²⁴视频画面时间区间: 01:08:30–01:08:50。内容: readelf 显示程序头里的 PT_INTERP 段; 讲师把 aarch64 改成 aarch65 后程序无法加载, 再 ln -s 一个同名文件, 程序又能跑了, 证明这仅仅是一个字符串。

动态链接的程序入口不在可执行文件里

一个动态链接的 ELF 文件，它的 program header 里有一个 PT_INTERP 段，内容是一个字符串——**解释器路径**（一般是 `/lib64/ld-linux-x86-64.so.2` 或 `/lib/ld-musl-*.so.1`）。
执行流程：

1. 内核的 `binfmt_elf.c` 加载主程序的 LOAD 段。
2. 同时加载 PT_INTERP 指定的解释器。
3. 把 PC 设置为**解释器的入口**，而不是主程序的 `_start`。
4. 解释器（本质上就是个普通用户态程序，叫 `ld.so`）先**加载主程序需要的所有.so**，做重定位，然后才**跳转到主程序的 `_start`**。

所以初次打断点在 `starti` 时，`info proc mappings` 里能看到 `a.out`、`[vdso]`、`[vvar]`、以及 `ld-linux`——但**看不到 `libc`**。只有跑到主程序 `_start` 时 `libc` 才被加载进来。

解释器是可以替换的

由于 PT_INTERP 就是个字符串，理论上你可以把**解释器换成任何东西**——甚至你自己写的一个“解释器”。加载器不检查它是否真的是标准 `ld-linux`。讲师演示：把字符串中的 `aarch64` 篡改成 `aarch65`，程序找不到文件而失败；再 `ln -s ld-musl-aarch64.so.1 ld-musl-aarch65.so.1`，程序又能跑——因为文件存在了。

8.2 PIC / PLT / GOT：运行时重定位的代价

动态链接把“填地址”推迟到运行时。由于同一份 `libc.so` 要映射到无数进程的不同虚拟地址，库代码本身必须是**位置无关的**（Position-Independent Code, PIC）。这通过 PLT / GOT 机制实现：

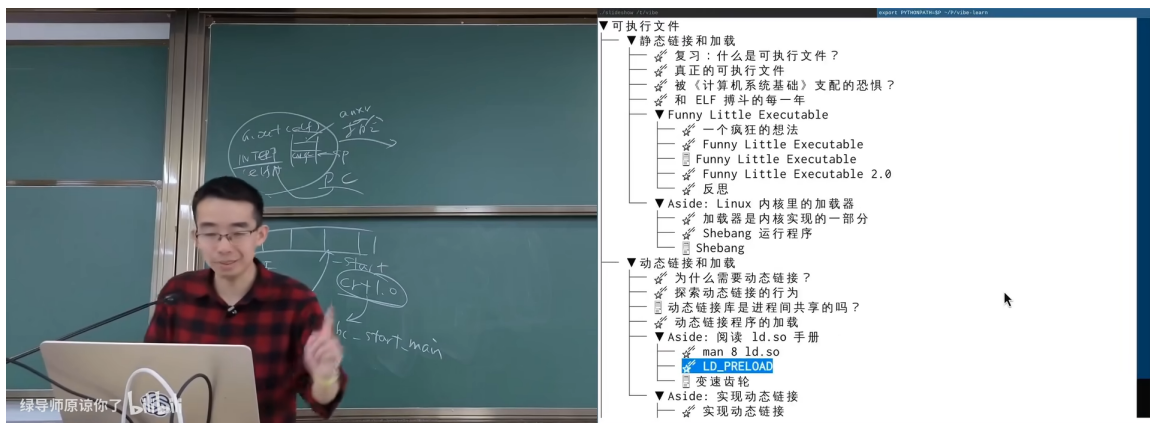


图 13: PLT / GOT 间接跳转示意：主程序调用 `printf@plt`，PLT 桩跳转到 GOT 中的函数地址；首次调用时 GOT 指向解析例程，解析后改写 GOT 指向真实 `printf`²⁶

²⁶视频画面时间区间：01:16:00–01:16:20。内容：slide 展示 PLT 和 GOT 的结构，讲解“任何跨库调用都要过一层间接跳转”带来的性能损耗。

动态链接的运行期开销：一层间接访问

无论是跨库调用还是跨库数据访问，动态链接都要经过一次 GOT 间接跳转。编译期你不知道 `printf` 的最终地址，只能编译成 `call printf@plt`；PLT 里的短桩再从 GOT 读出真实地址跳过去。

这意味着：

- 每次跨库调用多一次 load（访问 GOT 表）
- 代码体积略增（多一层桩）
- 可以通过编译器 hack（`-fvisibility=hidden`、symbol versioning）手动减少间接层

这就是为什么 C++ 模板密集的库、静态链接的小工具往往比动态链接版本更快一些。但对绝大多数工程场景，这点开销远小于空间节约和热升级带来的收益。

9 LD_PRELOAD：动态链接赋予的”超能力”

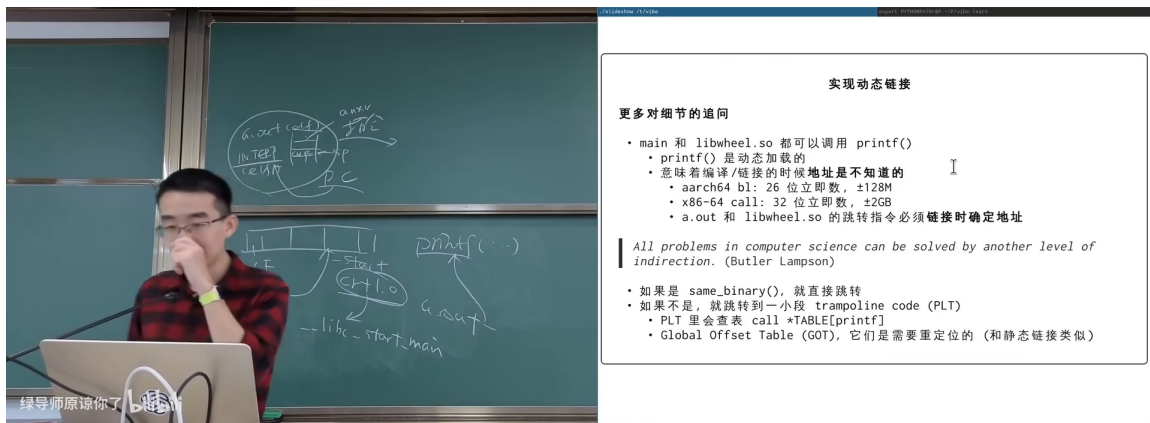


图 14: `LD_PRELOAD=./libwheel.so nsake`:把 `gettimeofday / clock_gettime / usleep / select / alarm` 等全部替换为 10 倍加速版，实现”变速齿轮”²⁸

动态链接解析顺序：LD_PRELOAD 优先于 libc

ELF 解释器 (ld.so) 解析符号的顺序：

1. 首先搜索 LD_PRELOAD 指定的.so
2. 然后是主程序声明的动态依赖 (DT_NEEDED)
3. 最后是系统库搜索路径 (`/lib`、`/usr/lib`、`$LD_LIBRARY_PATH`)

只要你在 LD_PRELOAD 指定的.so 里定义了与 libc 同名的符号，它就会覆盖 libc 的实现。

²⁸ 视频画面时间区间：01:22:15–01:22:45。内容：Makefile 编译 libwheel.so；`LD_PRELOAD=./libwheel.so nsake` 把贪吃蛇游戏加速 10 倍；`cmatrix` 快到 VS Code 都卡顿。

用 LD_PRELOAD 能做的花活

讲师列举（大都可由 AI 一键生成）：

- **变速齿轮**：hook `gettimeofday` / `clock_gettime` / `usleep` / `alarm` / `select` / `poll` 等时间相关调用，改变程序对“时间”的感知——老游戏加速、慢镜头调试、时间穿越测试。
- **malloc 监控**：hook `malloc` / `free`，记录所有分配，检测 use-after-free 或内存泄漏。
- **透视外挂（游戏）**：hook OpenGL 的 `glXSwapBuffers`，在画面提交前绘制额外 overlay 信息。
- **可控随机**：hook `rand()` / `random()`，让“抽奖程序”返回预设值——“联欢晚会抽奖程序理论上都不可信”。

共同原理：只要把一个函数的行为改掉，你就能改变整个程序的外在表现。这是动态链接给运维/调试/研究带来的底层杠杆。

10 总结与延伸

10.1 讲师的收束

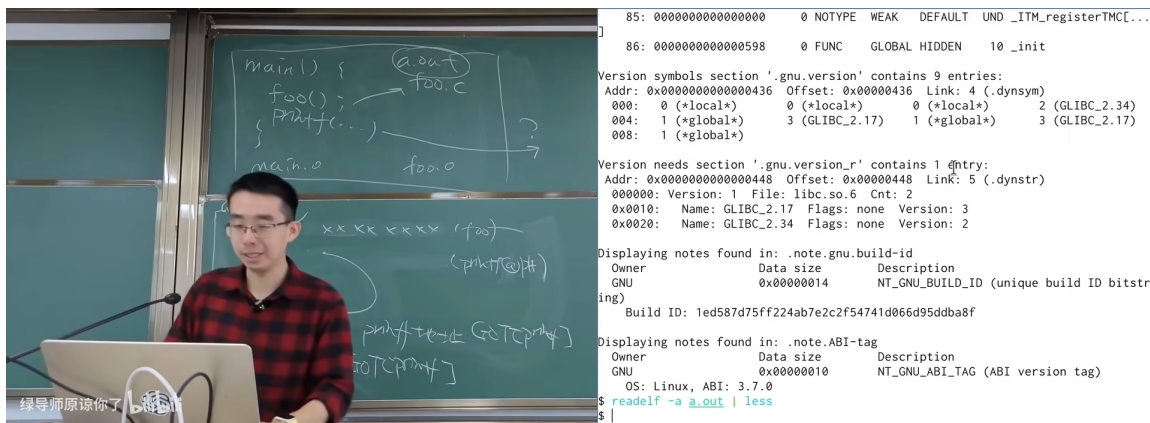


图 15: 一讲结束时的核心总结：链接和加载不过是“把数据结构搬进内存”，所有复杂性都是历史设计选择的累积³⁰

讲师在收束部分有几段颇值得记录：

- **“Intelligence is cheap”**：2026 年，AI vibe code 已经让“设计一个新格式、实现一个新加载器”成为一句话的工作。你只要能提出正确的问题、把握方向、并用 **critical thinking** 验证结论，剩下的交给 AI。
- **“程序的运行不会骗你”**：语言模型可能幻觉，但你让它生成的代码真的链接加载运行出 “Hello”，那就是真的 work。这是 AI 时代的 **critical thinking** 锚点。

³⁰ 视频画面时间区间：01:27:15–01:27:45。内容：讲师回到黑板前做最后总结，重申贯穿全课的基本原理——等价描述形式之间的自由转换。

- ”你们可能是最后一代大学生”：讲师认为在 AI 辅助下，课程的教法和学法都将发生根本性变化——这句话带着一丝怅然。

10.2 本讲的核心压缩

这一讲的”元认知”收获

如果把本讲压缩成三句话：

1. 可执行文件是进程初始状态的数据结构——所有 ELF 术语都只是这一事实在不同角度的投影。
 2. 链接 = 算数字填 offset；加载 = mmap 按权限搬段——把数据结构变换/搬运的过程，本质就是这两件事。
 3. 我们总可以在等价描述形式之间转换——Funny Little Executable / Markdown ELF / SVG ELF 都是 ELF。看清这一点，你就获得了重新设计一切教材的自由。
- 这一讲的真正价值不在记住 ELF 细节，而在获得”把复杂事物还原成数据结构”这种元能力。

10.3 拓展阅读与动手

- 源码：fs/binfmt_elf.c(ELF 加载器)、fs/binfmt_script.c(shebang 加载器)、arch/x86/entry/vdso/(vdso 实现)。用 AI Agent 按本讲的方法读，一个晚上能通读。
- 手册：System V ABI (<https://www.sco.com/developers/gabi/>)，至少读 Figure 3.9 的 Initial Process Stack 那一页。
- 动手：写一个 libwheel.so 的变种，hook rand() 让它总返回 6——验证”抽奖不可信”。5 行代码 + 一句 gcc -shared -fPIC。
- 延伸题：a.out 格式完整阅读 (CSAPP 或 Ulrich Drepper 的《How to Write Shared Libraries》)；了解 -Wl,-Bstatic/-Bdynamic、-fvisibility=hidden 等编译选项的实际效果。